

Technical Report: NIFTY-50 Investment Intelligence

Participant: Akshat

Problem Statement: Data-Driven Investment Intelligence Using NIFTY-50 Market Data

Submission Type: AI-powered investment intelligence dashboard using organizer-provided datasets

Prototype: Streamlit dashboard with analytics, prediction, portfolio construction, and explainability

1. Executive Summary

This project builds an investment intelligence platform for historical NIFTY-50 market data. The goal is not to create a black-box stock tip generator, but to transform raw daily market records into practical decision support for investors. The dashboard combines technical indicators, risk analytics, machine learning based next-day return prediction, portfolio construction for different risk profiles, and plain-English explanations.

The final prototype includes five main views:

- Market Overview for cross-stock comparison and sector context.
- Stock Analytics for detailed price, volume, risk, RSI, and MACD analysis.
- Predictor for next-trading-day return modeling and model-vs-baseline evaluation.
- Portfolio Builder for conservative, balanced, and aggressive allocations.
- Methodology for assumptions, limitations, and reproducibility notes.

The solution uses only the organizer-provided local CSV files and avoids live market data, financial APIs, news, social media sentiment, proprietary financial data, and alternative market datasets. All outputs are educational decision-support signals and should not be interpreted as financial advice.

2. Problem Understanding

Financial markets generate high-volume, noisy, and time-dependent data. Raw OHLCV records are useful, but they are not enough for a user who wants to compare opportunities, understand risk, and make portfolio-level decisions. A good investment intelligence system should therefore combine several layers:

- Historical stock performance analysis.
- Risk-adjusted performance measurement.
- Technical trend and momentum indicators.
- Forecasting or directional modeling with transparent validation.
- Portfolio recommendations aligned to user risk appetite.
- Clear explanations for why each insight or allocation was generated.

This project directly maps to the NIFTY-50 challenge by focusing on decision support rather than only price prediction. Prediction is included, but it is one component of a broader system that also evaluates risk, explains behavior, and constructs profile-specific portfolios.

3. Dataset and Exploratory Data Analysis

The implementation uses the organizer-provided NIFTY-50 files:

- `archive-2/NIFTY50_all.csv`
- `archive-2/stock_metadata.csv`

The aggregate market file is loaded through a validation layer that checks required columns, parses dates, converts numeric fields, removes unusable rows, and sorts records by symbol and date. The metadata file is joined to attach company and industry labels.

Dataset Coverage

Property	Value
Price rows loaded	235,192
Historical symbols	65
Metadata symbols	50
Date range	2000-01-03 to 2021-04-30
Industry categories in metadata	13
Trades missingness	48.8%
Historical symbols missing from latest metadata	16

The dataset spans more than two decades of market behavior. This makes it suitable for analyzing long-term returns, volatility regimes, drawdowns, and sector-level differences. Some symbols are historical names that are not present in the latest metadata file. Instead of dropping them, the app keeps those symbols and labels their industry as **Unclassified**. This preserves historical coverage while keeping the dashboard honest about metadata gaps.

EDA Observations

- The dataset includes multiple market cycles between 2000 and 2021, which is useful for stress-testing risk metrics.
- The **Trades** column has high missingness, so it is not used as a core model feature.
- Sector coverage is uneven. Financial Services, Energy, Consumer Goods, Automobile, Metals, and IT are better represented than smaller categories.
- Historical returns vary widely across stocks, so raw return alone can be misleading without volatility and drawdown context.
- Recent top Sharpe performers in the trailing 252-day window are concentrated in sectors such as Metals, IT, and selected industrial names, showing why sector context matters.

4. Feature Engineering

The feature pipeline converts raw historical prices into interpretable financial signals. Features are computed per symbol in chronological order to avoid leakage across stocks or time.

Technical Indicators and Derived Metrics

Feature	Purpose
Daily Return	One-day percentage price movement
Next Return	Prediction target for next-trading-day return
Return5 and Return20	Short-term and medium-term realized return signals
MA20, MA50, MA200	Trend-following moving averages
MA20 Distance and MA50 Distance	Price extension versus recent trend
Volatility20	Rolling 20-day return volatility
RSI14	Momentum/overbought-oversold indicator
MACD and MACD Signal	Trend and momentum crossover signal
Bollinger Upper/Lower Bands	Price range relative to 20-day volatility
Momentum20	20-day momentum signal
Volume Change	Change in traded volume

Feature	Purpose
Drawdown	Decline from cumulative return peak

These features are intentionally interpretable. They are common in financial analysis, allowed under the problem constraints, and easy to explain to evaluators and users. The project does not use external signals such as news sentiment, macroeconomic data, or live prices.

5. Methodology and Model Architecture

The predictor estimates next-trading-day return for the selected stock. The model is not presented as a guaranteed trading signal; it is used to add a quantitative forecasting layer to the decision-support dashboard.

Target

The prediction target is:

`Target Next Return = next trading day's daily return`

The app also derives predicted direction from the sign of the predicted return.

Model

The primary model is a `RandomForestRegressor` with:

- 160 trees
- maximum depth of 8
- minimum leaf size of 5
- fixed random state for reproducibility

Random Forest was selected because it can capture nonlinear relationships between technical indicators without requiring heavy preprocessing. It also provides feature importance values, which supports explainability.

If scikit-learn is unavailable, the project falls back to a NumPy linear regression model with the same interface. This keeps the project runnable even in constrained environments.

Validation Strategy

The split is chronological:

- First 80% of usable rows: training
- Final 20% of usable rows: testing

The data is never shuffled. This matters because financial time series have temporal order, and using future data to predict the past would produce misleading validation results.

Baseline

The model is compared against a simple naive baseline:

`Baseline prediction = previous day's return`

This prevents the dashboard from showing model metrics without context. A model is more useful when it can be compared to a simple rule.

6. Model Evaluation

The app reports:

- MAE
- RMSE
- R2
- Directional accuracy

For the default **RELIANCE** example, the model produced the following test metrics:

Metric	Random Forest	Previous-Day Baseline
MAE	0.014526	0.020820
RMSE	0.025914	0.036683
R2	-0.019861	-1.043644
Directional Accuracy	50.90%	51.28%

The Random Forest improves MAE and RMSE compared with the naive baseline, which means its return-size estimates are closer on average. Directional accuracy remains close to 50%, and R2 is slightly negative for **RELIANCE**, which reflects the inherent difficulty of short-horizon stock return prediction. This is why the project positions the predictor as one decision-support layer, not as a standalone buy/sell engine.

Top Feature Importances for **RELIANCE**

Feature	Importance
Daily Return	0.1682
Return5	0.1357
Volatility20	0.1127
MA20 Distance	0.0811
Volume Change	0.0714
MA50 Distance	0.0707
RSI14	0.0693
MACD Signal	0.0635

The importance pattern is intuitive: recent returns, rolling volatility, moving-average distance, and volume behavior are useful short-horizon signals. The dashboard exposes these values so the user can inspect model behavior rather than treating predictions as unexplained outputs.

7. Risk Assessment Methodology

The risk module evaluates both individual stocks and portfolio candidates. All risk metrics are computed from daily returns.

Metric	Meaning
Annualized Return	Compounded return scaled to a 252-trading-day year
Annualized Volatility	Standard deviation of daily returns scaled by $\sqrt{252}$

Metric	Meaning
Sharpe Ratio	Excess annualized return divided by annualized volatility
Sortino Ratio	Excess annualized return divided by downside volatility
Maximum Drawdown	Largest peak-to-trough decline over the period

The Sharpe and Sortino calculations use a 5% annual risk-free rate. These risk metrics are displayed in the stock analytics tab and reused in the portfolio builder.

For example, the full-period RELIANCE risk summary is:

Metric	Value
Annualized Return	10.33%
Annualized Volatility	38.15%
Sharpe Ratio	0.14
Sortino Ratio	0.16
Maximum Drawdown	-79.01%

This example shows why risk-adjusted analysis matters. A stock can have positive long-term return while still experiencing large drawdowns.

8. Portfolio Construction Logic

The portfolio module creates allocations for three investor profiles:

- Conservative
- Balanced
- Aggressive

Each profile uses trailing 252 trading days where available. Stocks must have enough observations to avoid recommendations based on very small histories. Candidate stocks are scored using normalized financial metrics. Weights are normalized to 100% and capped by profile to reduce single-stock concentration.

Profile Scoring

Profile	Scoring Emphasis	Weighting Behavior
Conservative	Sharpe, low volatility, drawdown control	Inverse-volatility adjusted weights
Balanced	Sharpe, return, low volatility, momentum	Score-proportional weights
Aggressive	Return, momentum, Sharpe, drawdown awareness	Higher tolerance for momentum and volatility

The profile logic is intentionally explainable. Every selected stock receives a short reason that references its volatility, return, Sharpe ratio, momentum, or drawdown profile.

Example Portfolio Outputs

Profile	Largest Weight	Weighted Return	Weighted Volatility
Conservative	14.7%	205.7%	36.5%
Balanced	16.9%	241.9%	40.4%
Aggressive	22.1%	262.4%	42.9%

These values come from the trailing 252-day window in the local dataset. The aggressive portfolio accepts more volatility in exchange for stronger return and momentum exposure, while the conservative portfolio reduces concentration and emphasizes risk-adjusted behavior.

Example top holdings:

Profile	Example Holdings
Conservative	INFY, GRASIM, TELCO, JSWSTEEL, WIPRO
Balanced	JSWSTEEL, TATASTEEL, BAJAUTOFIN, HINDALCO, WIPRO
Aggressive	JSWSTEEL, TATASTEEL, BAJAUTOFIN, WIPRO, HINDALCO

The allocations are not recommendations to invest real money. They demonstrate how historical returns, risk, and momentum can be transformed into a transparent profile-based portfolio construction workflow.

9. Explainability and Transparency

Explainability is built into the user experience in four ways:

1. The predictor displays feature importance, model metrics, and baseline comparison.
2. The market overview shows a risk-return scatter plot for cross-stock comparison.
3. The portfolio builder gives a plain-English explanation for every holding.
4. The methodology tab states assumptions, validation strategy, and limitations.

This design helps evaluators understand why the system generated an insight. It also reduces the risk of overclaiming. For example, weak or noisy prediction performance is still visible to the user through MAE, RMSE, R2, and directional accuracy.

10. Working Prototype and User Flow

The working prototype is implemented as a Streamlit dashboard. It can be run locally with:

```
streamlit run app.py
```

The main user flow is:

1. Select a stock symbol and date range in the sidebar.
2. Review market-level risk and industry context in Market Overview.
3. Inspect stock-specific price trend, volume, RSI, MACD, and risk metrics.
4. Open Predictor to view model metrics, actual vs predicted returns, and feature importance.
5. Select an investor profile and review the generated allocation in Portfolio Builder.
6. Read the methodology tab for assumptions and limitations.

The dashboard is dense and functional rather than decorative. It prioritizes charts, tables, KPIs, and clear controls because the problem is analytical and decision-support oriented.

11. Reproducibility and Repository Structure

The repository is organized so that the results can be reproduced from the local CSV files:

File or Folder	Purpose
<code>app.py</code>	Streamlit dashboard
<code>nifty_intel/data.py</code>	CSV loading, validation, date parsing, metadata joins
<code>nifty_intel/indicators.py</code>	Technical indicators and derived features
<code>nifty_intel/risk.py</code>	Risk metrics and stock summaries
<code>nifty_intel/prediction.py</code>	Prediction model, baseline, and evaluation
<code>nifty_intel/portfolio.py</code>	Profile-based portfolio construction
<code>scripts/smoke_test.py</code>	End-to-end smoke test
<code>requirements.txt</code>	Dependency configuration
<code>models/README.md</code>	Explains runtime model training and artifact policy
<code>README.md</code>	Setup, run, dataset placement, and reproducibility instructions

The model trains at runtime from the provided local CSV files. No serialized model binary is committed because the selected stock and date window can change inside the dashboard, and a stale model file would be less transparent than a reproducible training pipeline.

The smoke test verifies:

- data loading;
- indicator generation;
- risk metric calculation;
- prediction model execution;
- portfolio construction and normalized weights.

12. Constraint Compliance

The project respects the official constraints.

Allowed techniques used:

- Feature engineering
- Statistical transformations
- Technical indicators
- Machine learning model
- Portfolio optimization/scoring techniques

Not used:

- Live market data
- Financial APIs
- News datasets
- Social media sentiment data
- Proprietary financial data
- Alternative market datasets

The project only uses the local organizer-provided NIFTY-50 files listed in the README. Dataset folders are intentionally ignored by Git so the public repository remains lightweight while still documenting exact local file placement.

13. Key Insights

The project produced several useful observations:

- Risk-adjusted metrics are more informative than raw return when comparing stocks across sectors.
- Short-horizon stock return prediction is noisy, so model outputs should be paired with baselines and risk metrics.
- Profile-based portfolio construction is more useful than a single generic ranking because investor risk tolerance matters.
- Explainability improves trust: feature importance and per-holding reasons make the system easier to audit.
- Historical metadata can be incomplete for old symbols, so fallback labels are necessary for robust dashboards.

14. Limitations and Future Work

Limitations:

- The dataset ends on 2021-04-30, so outputs are historical and not current market recommendations.
- No live prices, financial APIs, news, sentiment, macroeconomic data, proprietary data, or alternative market datasets are used.
- The predictor focuses on next-day returns, which are inherently noisy.
- The dashboard does not perform full walk-forward backtesting.
- Sector exposure constraints are simple and can be expanded.

Future improvements:

- Add walk-forward validation across multiple market regimes.
- Add full portfolio backtesting with transaction cost assumptions.
- Add sector and concentration constraints for more realistic allocation control.
- Compare Random Forest with gradient boosting and time-series models.
- Add exportable portfolio reports from inside the dashboard.
- Add anomaly detection for volatility spikes, unusual volume, and extreme drawdowns.

15. Conclusion

The project delivers a complete investment intelligence MVP for the NIFTY-50 problem statement. It includes a working dashboard, technical indicators, machine learning prediction, baseline evaluation, risk analytics, explainable portfolio construction, documentation, and a PDF report. The solution is reproducible, constraint-compliant, and focused on practical decision support rather than unsupported financial claims.